

# MemPatch: Evidential Revision Scaffolds for Rapid Memory Integration in LLM Agents

Anonymous submission

## Abstract

Persistent-memory large language model (LLM) agents use stored records to guide future retrieval and action, yet corrective evidence can invalidate a record without removing its license for downstream reuse. Answer accuracy alone can therefore hide a corrupted persistent state. We introduce MemPatch-Bench, a benchmark for post-admission memory revision with executable reference semantics, record-level validity labels, and a development-calibrated diagnostic profile. We also introduce MemPatch, a parameter-free mediation layer that compiles model proposals into typed patches, checks them against a deterministic Revision Guard, and emits a replayable commit certificate. Under matched frozen backbones, canonical projection yields 100% final-response schema compliance while raw action parse validity ranges from 87.8% to 100%. MemPatch also stabilizes memory-state prediction, including 84.7% state accuracy on a backbone whose strongest frozen baseline reaches 9.3%. The gains are not uniform across every answer-level metric: structured mediation can trade proposal recall for state integrity. We separate guarantees of the deterministic mediator from empirical properties of the neural proposer, yielding an evaluation and runtime framework for memory updates that are explicit, evidence-grounded, and externally auditable.

## Introduction

Large language model (LLM) agents are increasingly deployed in long-horizon environments where they must maintain state across extended temporal trajectories. Rather than relying solely on transient context windows, modern agent architectures employ persistent memory stores to accumulate experiences, document tool executions, and record intermediate planning states. These stored records actively guide subsequent system behaviors, serving as the knowledge base for downstream retrieval, planning, tool use, and database writes. In this persistent setting, a critical and under-addressed risk is record obsolescence—where a persistent record remains licensed for future reuse after corrective, conflicting, superseding, or policy-relevant evidence has arrived.

Retrieval alone does not solve this problem. For example, a calendar agent may persistently store that a user has a meeting scheduled on Tuesday. When subsequent email evidence reveals that the meeting has been moved to Friday, the agent must update its memory state. If the agent retrieves the new email and answers the immediate query correctly

while leaving the contradicted Tuesday record active in its store, subsequent tasks may retrieve the stale Tuesday record, leading to scheduling conflicts. This silent failure permits immediate query success but leaves a contaminated state that propagates errors in future execution turns.

To isolate and evaluate this failure mode, we present MemPatch-Bench, a diagnostic benchmark specifically designed to evaluate post-admission validity revision in persistent agent memory. MemPatch-Bench requires agents to maintain record-level validity states under chronological evidence streams, scoring updates against hidden ground-truth labels. It evaluates agents not merely on their downstream answers, but on their multi-channel diagnostic profile, auditing error signatures across task decisions, memory validity states, evidence citations, failure diagnoses, and answers.

To address this challenge without requiring parameter updates, we propose MemPatch, a parameter-free plug-in runtime layer that separates opaque semantic proposals from verifiable persistent-memory transitions. MemPatch does not claim access to a model’s latent reasoning or guarantee that its proposal is semantically correct. Instead, it confines black-box inference to proposal generation and makes every committed transition explicit, policy-checked, attributable, and replayable. A normalized adapter compiles heterogeneous model outputs into typed actions; a deterministic Revision Guard validates references, transition legality, conflicts, and evidence authorization before canonical commit.

Empirical evaluations with frozen open-weight backbones show that prompting, retrieval-augmented generation (RAG), and summary-memory baselines exhibit sharply different failure signatures that are obscured by a single complete-case score. Canonical projection gives MemPatch uniform final-response schema compliance across the completed backbones, while raw action parse validity remains model dependent. MemPatch also stabilizes record-level state prediction, most visibly when direct prompting fails to express the benchmark state contract. These results support a narrower but stronger claim: the mediator controls how proposed changes reach persistent state; it does not replace the semantic competence of the proposer.

This paper makes the following co-equal primary contributions:

1. **MemPatch-Bench:** A controlled benchmark with chronological evidence, five record-validity states, exe-

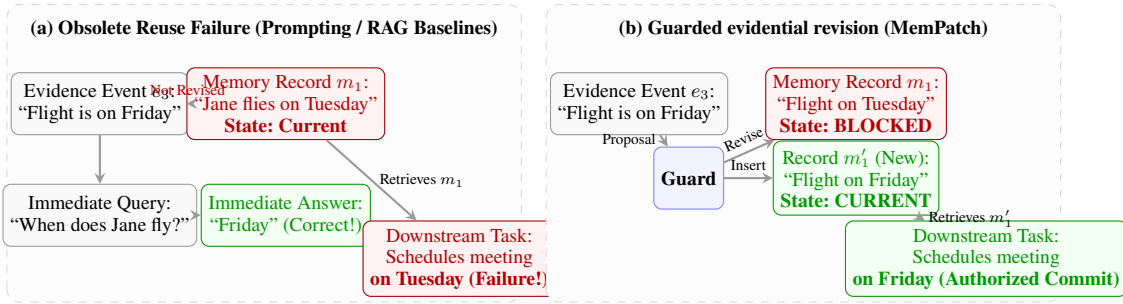


Figure 1: Why answer correctness is insufficient. (a) A system can answer from new evidence while an obsolete record remains licensed for later reuse. (b) MemPatch makes the proposed revision explicit and restricts downstream reuse to records committed as CURRENT by the Guard.

cutable reference transitions, and a seven-channel diagnostic profile calibrated exclusively on development data.

2. **MemPatch:** A plug-in mediation layer that converts heterogeneous model outputs into typed, evidence-grounded patches and exposes a deterministic authorization and commit boundary.
3. **Separated Evidence:** Matched frozen-backbone experiments, controlled corruptions, and replay audits that distinguish proposer accuracy from mediator guarantees instead of collapsing both into one score.

## Related Work

**Memory Storage, Admission, and Retrieval.** Existing architectures focus on memory storage structures, retrieval models, and periodic summarization or compression. Works such as MemGPT manage memory via paging and swapping between a context window and external storage (Packer et al. 2024). Mem0 provides CRUD-like operations over natural-language facts (Chhikara et al. 2025), while systems like A-MEM and MemoryOS focus on hierarchical organization, node consolidation, and context length reduction (Xu et al. 2025; Kang et al. 2025). More recent methods learn memory operation selection or adaptive structures (Yu et al. 2026; Lu et al. 2026; Xu et al. 2026; Zhang et al. 2026). Unlike these frameworks which optimize storage allocation or retrieval performance, MemPatch-Bench and MemPatch focus strictly on *post-admission validity revision*—validating whether existing memory records remain logically active or superseded as new evidence chronologically arrives.

**Memory Benchmarks.** Standard evaluations assess long-term retrieval, QA over extended chat histories, or task success in evolving virtual environments. The LoCoMo benchmarks evaluate conversational memory recall over tens of thousands of tokens (Maharana et al. 2024; Li et al. 2026). The LONGMEMEVAL family assesses temporal updates and experience acquisition in dialogue systems (Wu et al. 2025, 2026). Other benchmarks study memory structure or self-evolving retrieval performance (Tan et al. 2025; Shutova et al. 2026; Wang et al. 2026; Hu, Wang, and McAuley 2026). In contrast to evaluating memory through downstream answer accuracy (which can mask invalid memory states),

MemPatch-Bench scores the record-level validity mapping directly, tracking evidence grounding and multi-dimensional error metrics.

**Structured Inference and Runtime Enforcement.** Inference-time scaffolds structure language model reasoning without parameter updates. ReAct interleaves reasoning and actions (Yao et al. 2023b), while Reflexion and Self-Refine implement feedback loops for self-correction (Shinn et al. 2023; Madaan et al. 2023). Reasoning pathways are generalized into thoughts in TREE OF THOUGHTS and GRAPH OF THOUGHTS (Yao et al. 2023a; Besta et al. 2024; Zhou et al. 2024; Luo et al. 2026). Embodied agents like Voyager execute neural code proposals within deterministic environment engines (Wang et al. 2023). MemPatch builds on these paradigms by splitting opaque neural generation from deterministic, symbolic state updates. However, we apply this separation specifically to memory revision, routing neural proposals through a symbolic Guard to verify references and resolve conflicts.

## Problem Formulation

We formalize the problem of post-admission memory validity revision. Let a scenario be defined as a tuple:

$$x = (M, E, q, c), \quad (1)$$

where  $M = \{m_i\}_{i=1}^n$  is the persistent memory store,  $E = \{e_j\}_{j=1}^k$  is the chronological evidence ledger,  $q$  is the downstream query, and  $c$  contains public policy and scope constraints. Both memory records and evidence events are associated with unique, stable identifiers and natural-language payloads.

The public revision view is constructed via a deterministic function  $B$ :

$$V = B(M, E, q, c, s^0, \Gamma), \quad (2)$$

where  $s^0$  is the initial state map of the memory records, and  $\Gamma$  is the published transition policy. Crucially,  $V$  excludes gold labels, private metadata, and evaluator-specific information, exposing only the candidate records, evidence ledger, and active policies to the agent.

The memory state space  $\Sigma$  is defined over five distinct epistemic classes:

$$\Sigma = \{\text{Current}, \text{Blocked}, \text{Unresolved}, \text{OutOfScope}, \text{ShouldNotStore}\}. \quad (3)$$

The downstream reuse license is a function  $\lambda : \Sigma \rightarrow \{0, 1\}$ :

$$\lambda(z) = \mathbf{1}[z = \text{Current}], \quad (4)$$

which dictates that a memory record  $m_i$  is licensed for future reuse in downstream tasks if and only if its validity state  $s_i$  is Current. Records with other states (e.g., blocked due to unmet dependencies, unresolved due to conflicting evidence, contextually out-of-scope, or excluded by safety policy) are restricted from reuse.

**Output Contract.** The agent’s output is structured as a tuple:

$$r = (d, s, Z, f, a), \quad (5)$$

where  $d$  is a downstream decision (e.g., use memory, escalate, ask clarification, refuse due to policy),  $s : M \rightarrow \Sigma$  is the final memory validity mapping,  $Z \subseteq I_E$  is the response-level evidence set,  $f \in \mathcal{F}$  is the failure diagnosis, and  $a$  is the final generated answer. For a committed patch, provenance is finer grained:  $\hat{Z} : M \rightarrow 2^{I_E}$  associates each changed record with its own evidence witness.

**Diagnostic Measurement Model.** To assess agent performance without hiding failure modes inside a single conjunction, we define seven response channels: decision, state, evidence, minimality, diagnosis, answer, and schema. They correspond to decision accuracy, memory-state accuracy, evidence  $F_1$ , minimal-evidence exact match, diagnosis accuracy, answer key-fact accuracy, and schema compliance. We probe these channels with seven controlled corruptions  $\mathcal{O}$ : wrong decision, wrong state, wrong evidence, overcitation, wrong diagnosis, malformed schema, and missing answer. Trace audit is evaluated separately because replayability is a property of an executed transition, not of a five-field response.

For each scenario, we define an error vector  $\mathbf{e}(r, g) = (\ell_k(r, g))_{k \in \mathcal{K}}$ , where each  $\ell_k$  measures the correctness of output channel  $k$  against gold label  $g$ . The diagnostic profile of an agent  $h$  is the expected error vector:

$$\mathbf{R}(h) = \mathbb{E}_{\xi}[\mathbf{e}(h(B(\xi)), g(\xi))]. \quad (6)$$

The sensitivity matrix  $A = (a_{kj}) \in \mathbb{R}^{7 \times 7}$  records the decrement in metric  $k$  under corruption  $o_j$ . All operators, mappings, and calibration constants are fixed exclusively on the L3 development split before L4 evaluation. Full rank is an empirical audit of this chosen corruption basis, not a claim that all real model failures are linearly separable. We report the complete error vector as the primary outcome. Joint revision success,  $J(\mathbf{b}) = \prod_k (1 - b_k)$ , is retained only as a secondary complete-case statistic because it maps many distinct error signatures to the same zero value.

## MemPatch-Bench

We introduce MemPatch-Bench, a diagnostic evaluation suite for post-admission memory validity revision. Unlike existing evaluations that assess memory quality through downstream task success, MemPatch-Bench directly audits the record-level validity state of the agent’s memory store.

**Dataset Composition and Statistics.** The dataset contains a total of 4,000 scenarios:

- **L3 Development Split** (3,500 cases): Used for prompt engineering, schema validation, Guard testing, controlled-corruption calibration, and the fixed train/validation partition for adaptation.
- **L4 Held-Out Split** (500 cases): Disjoint in templates and domains from L3, serving as the sole official evaluation split.

The scenarios span six diverse domains (software engineering, task management, data analysis, enterprise tooling, customer support, and scientific work) and are structured using stable, semantic-free identifiers.

**Seven Memory Integration Failure Modes.** Each scenario in MemPatch-Bench targets one of seven structural memory integration failures:

1. **Stale-Reuse:** Obsolete records remain licensed for downstream reuse after contradictory evidence arrives.
2. **Under-Update:** Failure to identify and modify records targeted by valid updates.
3. **Conflict Collapse:** Concurrent contradictory updates are committed as if resolved instead of being represented as unresolved.
4. **Scope Leakage:** Contextual or domain-specific exclusions are ignored, leaking out-of-scope records into active reasoning.
5. **Wrong-Source Attribution:** Crediting updates to incorrect or ungrounded evidence events.
6. **Policy Violation:** Violating safety, retention, or privacy constraints during memory modification.
7. **Memory Hallucination:** Proposing modifications to records that do not exist or using non-existent evidence identifiers.

**Evaluation Protocol.** The evaluation protocol strictly isolates hidden gold labels from the agent. A deterministic evaluator scores  $r = (d, s, Z, f, a)$  against the state produced by executable reference semantics. Proposition ensures that a fixed scenario program has one reproducible evaluator output; it does not claim that the natural-language scenario generator is error free.

To account for intra-family dependence, the 500 L4 instances are grouped into 18 template-family clusters spanning all seven failure modes. Confidence intervals are obtained by paired cluster bootstrap: each replication resamples whole template families with replacement and preserves all instances within a sampled family. We report the number of replications and the interval construction rule with the full metrics; we do not treat bootstrap intervals as distribution-free guarantees.

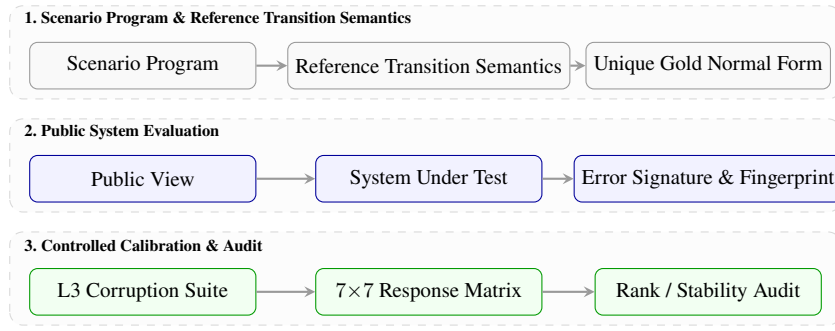


Figure 2: MemPatch-Bench protocol. Executable reference semantics produce deterministic gold states. Systems see only the public view and are scored by a diagnostic vector. The corruption suite and response matrix are fixed on L3; L4 is touched only for final evaluation.

**Metamorphic Equivariance Verification.** We audit the reference implementation under three semantics-preserving transformations: record-ID bijection, irrelevant-event insertion, and distractor reordering. Across all 500 L4 scenarios, the executable reference semantics exhibits zero defect for each transformation,  $\Delta_{\text{ref}}(g) = 0$ . For a model or mediator  $f$ , we instead measure  $\Delta_f(g) = d(f(gx), gf(x))$ ; nonzero defect is a robustness result, not an invalidation of the evaluator.

## MemPatch Runtime

The MemPatch runtime is a parameter-free plug-in layer designed to convert predictions from frozen or adapted models into policy-checked, attributable, and replayable memory patches. It separates neural prediction from deterministic runtime verification.

### Plug-in Interface and Compilation

The MemPatch runtime accepts semantic proposals without editing model parameters. It normalizes heterogeneous outputs through an adapter contract  $A_\Pi$ :

$$A_\Pi : Y_\Pi \rightarrow (\hat{s}, \hat{Z}), \quad (7)$$

where  $Y_\Pi$  is the raw generation,  $\hat{s} : M \rightarrow \Sigma$  is the proposed state map, and  $\hat{Z} : M \rightarrow 2^{I_E}$  supplies record-level provenance. The compiler constructs  $\tilde{\alpha} = C_V(s^0, \hat{s}, \hat{Z})$ . An empty or rejected patch is a *no-commit* outcome: it prevents an unauthorized write but is not claimed to repair an already stale state.

### Revision Guard

To prevent ungrounded or contradictory actions from modifying memory, compiled action sequences are routed through the symbolic Revision Guard  $G_\Gamma$ :

$$(\alpha^+, \rho) = G_\Gamma(V, \tilde{\alpha}), \quad (8)$$

where  $\alpha^+$  is the admitted canonical patch, and  $\rho$  is the rejection log detailing the failure reasons. The Guard filters

proposals based on a transition policy  $\Gamma$  containing five predicates:

$$\begin{aligned} \mathcal{L}_\Gamma(V) = \{ \alpha \mid & \text{Schema}(\alpha) \wedge \text{References}_V(\alpha) \\ & \wedge \text{Transitions}_\Gamma(\alpha) \wedge \text{NoConflict}(\alpha) \\ & \wedge \text{Authorized}_\Gamma(\alpha) \}. \end{aligned}$$

- $\text{Schema}(\alpha)$ : Checks that each action matches the required structure.
- $\text{References}_V(\alpha)$ : Ensures all target and replacement identifiers exist in the public memory  $M$  exposed by  $V$ .
- $\text{Transitions}_\Gamma(\alpha)$ : Verifies that all state changes follow valid transitions.
- $\text{NoConflict}(\alpha)$ : Rejects overlapping or contradictory actions within the same patch.
- $\text{Authorized}_\Gamma(\alpha)$ : Verifies that the cited evidence identifiers match the transition rule triggers registered in  $\Gamma$ .

Admitted patches are projected to yield candidate state maps  $s^* = \Delta(s^0, \alpha^+)$ . Every transition outputs an audit certificate trace:

$$\tau = (H(V), H(\Pi), \hat{r}, \tilde{\alpha}, \alpha^+, \rho, s^0, s^*), \quad (9)$$

which logs the input view hash  $H(V)$ , predictor identity, raw proposal, candidate and admitted actions, rejection reasons, and pre/post states.

### Formal Guarantees

We separate properties of the benchmark reference executor from properties of the runtime mediator.

**Proposition 1 (Deterministic Reference Execution).** *Let  $E = (e_1, \dots, e_T)$  be a finite, totally ordered event trace. If the reference step function  $F_\Gamma : \Sigma_{\text{cfg}} \times E \rightarrow \Sigma_{\text{cfg}}$  is total and deterministic, then the evaluator returns a unique configuration*

$$C^* = F_\Gamma(\dots F_\Gamma(F_\Gamma(C_0, e_1), e_2) \dots, e_T). \quad (10)$$

The result follows by induction on  $T$ . The implementation additionally enumerates all  $\binom{23}{2} = 253$  unordered rule pairs in a synthetic order-swap stress test. This audit is reported as implementation coverage, not as a critical-pair confluence proof.

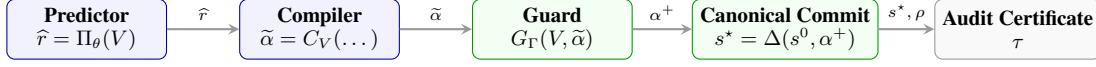


Figure 3: MemPatch as a plug-in mediation layer. Model-specific output is confined to the predictor; compilation, authorization, commit, and certificate generation are deterministic.

### Proposition 2 (Conditional Diagnostic Identifiability).

Under the local linear model  $\Delta = A\mathbf{w} + \varepsilon$ , if the L3-calibrated matrix  $A \in \mathbb{R}^{7 \times 7}$  has full column rank, the noiseless coefficients in the chosen corruption basis are unique:  $\mathbf{w} = A^\dagger \Delta$ . Moreover,

$$\|\hat{\mathbf{w}} - \mathbf{w}\|_2 \leq \|A^\dagger\|_2 \left( \|\hat{\Delta} - \Delta\|_2 + \|\hat{A} - A\|_2 \|\mathbf{w}\|_2 \right). \quad (11)$$

This identifies mixtures only within the seven controlled operators; it does not establish completeness of that basis for organic failures. Calibration and rank auditing use L3 only.

**Theorem 1 (Guarded Mediation).** Assume (A1)  $s^0 \models \mathcal{I}_\Gamma$ ; (A2) the Guard admits only  $\alpha \in \mathcal{L}_\Gamma(V)$ ; (A3) every legal action preserves  $\mathcal{I}_\Gamma$ ; and (A4) each admitted action records a complete transition witness in  $\tau$ . Then:

- **Mutation confinement:** only records targeted by admitted actions may change;
- **Invariant preservation:**  $s^* \models \mathcal{I}_\Gamma$ ;
- **Replayable mediation:** deterministic replay of  $(s^0, \tau)$  yields  $s^*$ ;
- **Trace-complete authorization:** every committed mutation has a record-level evidence witness.

These are mediator properties conditional on A1–A4; they do not imply semantic truth or proposal completeness.

**Corollary 1 (Deterministic Replay and Idempotency).** For deterministic parsing, Guard evaluation, reconciliation, and projection, replaying the same audit trace  $\tau$  satisfies  $\text{Replay}(s^0, \tau) = s^*$ . Replaying the same canonical patch  $\alpha^*$  is idempotent:  $\Delta(\Delta(s, \alpha^*), \alpha^*) = \Delta(s, \alpha^*)$ . **Implementation Artifact:** `mempatch/tests/test_consensus_mode.py`

**Corollary 2 (Local Noninterference).** If neither an admitted action nor an authorized reconciliation decision targets  $m_i$ , then  $s_i^* = s_i^0$ . Malformed references and rejected actions therefore cannot silently alter unrelated records. **Implementation Artifact:** `mempatch/revision/runtime/dpa_runtime.py` (tested in `mempatch/tests/test_dpa_kernel.py`)

**Verifiability Metric.** To detect logging or mediation defects empirically, the system calculates the *Externally Verifiable Transition Fraction* (EVTF):

$$\text{EVTF} = \frac{\sum_i \mathbf{1}[s_i^* \neq s_i^0] \mathbf{1}[\text{VerifyWitness}(i, \dots)]}{\sum_i \mathbf{1}[s_i^* \neq s_i^0]}. \quad (12)$$

We report EVTF only for runs that emit the required per-transition certificate; it is not imputed for ordinary prompting baselines.

## Experiments

Our principal comparison holds the backbone and adaptation status fixed: all systems in Table 1 use frozen base models without benchmark-specific training. We separately use a shared-checkpoint adaptation study to diagnose proposal-interface effects; it is not mixed into the frozen comparison.

**Data and Protocol.** MemPatch-Bench contains 3,500 L3 development scenarios and 500 template-disjoint L4 test scenarios, covering 15,203 memory records, 28,409 chronological events, and seven failure modes. Prompts, corruption operators, and calibration constants are fixed on L3. L4 is used once for final reporting. Every system must produce the same five-field response contract and is scored by the same deterministic evaluator. Invalid parses and schema violations are retained as failures rather than silently repaired.

**Systems.** The five frozen baselines are: *Frozen Direct Prompting*, which predicts the final response from the task view; *Full Context*, which exposes the complete public scenario; *Lexical RAG*, which retrieves evidence by lexical relevance; *Time-Aware RAG*, which adds recency-aware ranking; and *Summary Memory*, which compresses prior events. MemPatch uses the same frozen backbone to propose typed patch actions, followed by the Revision Guard and canonical projection. Thus the comparison changes the inference interface and mediator, not model weights.

**Persistent-State Reliability.** Across the three completed frozen backbones, MemPatch raises average memory-state accuracy from the strongest baseline average of 56.1% to 83.8%. The largest difference occurs on Mistral-Nemo: the strongest baseline reaches only 9.3%, while typed patches followed by deterministic projection reach 84.7%. This is not merely an average dominated by one model: MemPatch is best on Qwen and Mistral and remains competitive with the strongest Phi-4 baseline (82.9% versus 85.1%).

**Interface Robustness.** All projected MemPatch responses satisfy the five-field output schema, whereas baseline compliance ranges from 0% to 100% and collapses on Mistral. This 100% is a property of deterministic canonical projection, not evidence that every neural proposal is well formed: raw action parse validity is 90.4% for Qwen, 87.8% for Phi-4, and 100% for Mistral. The result isolates a practical role for mediation: model-specific surface forms are normalized before persistent state is committed. The Guard cannot make an omitted proposal correct, but it can prevent malformed output from becoming an implicit write protocol.

**Diagnostic Trade-offs.** The full vectors prevent overclaiming. MemPatch does not dominate every response channel. On Qwen, for example, frozen direct prompting has higher decision  $F_1$  and evidence  $F_1$ , while MemPatch has

| System                  | Memory-State Accuracy (%) |             |             |             | Final-Response Schema Compliance (%) |              |              |              |
|-------------------------|---------------------------|-------------|-------------|-------------|--------------------------------------|--------------|--------------|--------------|
|                         | Qwen                      | Phi-4       | Mistral     | Avg.        | Qwen                                 | Phi-4        | Mistral      | Avg.         |
| Frozen Direct Prompting | 82.2                      | 74.0        | 0.0         | 52.1        | 100.0                                | 59.6         | 0.0          | 53.2         |
| Full Context            | 74.6                      | 74.1        | 2.9         | 50.5        | 100.0                                | 63.8         | 4.2          | 56.0         |
| Lexical RAG             | 79.9                      | 79.2        | 9.3         | 56.1        | 90.6                                 | 58.2         | 1.8          | 50.2         |
| Time-Aware RAG          | 76.4                      | <b>85.1</b> | 6.5         | 56.0        | 98.4                                 | 63.0         | 11.0         | 57.5         |
| Summary Memory          | 81.1                      | 81.3        | 0.0         | 54.1        | 76.2                                 | 54.4         | 0.0          | 43.5         |
| MemPatch                | <b>83.9</b>               | 82.9        | <b>84.7</b> | <b>83.8</b> | <b>100.0</b>                         | <b>100.0</b> | <b>100.0</b> | <b>100.0</b> |

Table 1: Matched frozen-backbone comparison on the 500-case L4 test set. No row uses LoRA or benchmark-specific adaptation. Schema compliance is measured on the final five-field response; for MemPatch, this response is produced by deterministic canonical projection. Full diagnostic vectors are reported separately.

higher memory-state accuracy and a schema-valid projected response. On Phi-4, MemPatch obtains the strongest diagnosis accuracy (37.4%) but trails Time-Aware RAG in decision  $F_1$ . These differences support the benchmark design: answer-level, evidence-level, and persistent-state correctness are related but not interchangeable.

**Why Joint Is Secondary.** Joint revision success requires every scored component to be correct for the same case. It is useful as a strict complete-case statistic, but a zero does not distinguish malformed output, a wrong diagnosis, or a single evidence mismatch from a wrong persistent state. We therefore retain Joint in reproducibility artifacts rather than using it as the headline measure.

### Mediator and Adaptation Diagnostics

**Guard Effect.** For frozen Qwen, Phi-4, and Mistral, the Guarded path improves memory-state accuracy over direct projection of the same typed-action proposal by 4.9, 3.2, and 0.8 percentage points, respectively. The small or zero differences on other channels are expected: the Guard authorizes and projects proposed transitions; it is not a second semantic predictor. Controlled invalid-action injection is the appropriate setting for measuring rejection and unauthorized write-through.

**Shared-Checkpoint Adaptation.** An auxiliary QLoRA experiment trains one multitask adapter per backbone on both final-state and patch-action targets. Checkpoints are saved every 128 steps through step 1,024 and selected solely by minimum L3 validation loss. Final-State Control and MemPatch then use the exact same selected adapter, preventing interface-specific checkpoint selection. On the completed Qwen run, Final-State Control achieves higher complete-case accuracy, while MemPatch yields explicit action validity and per-transition audit coverage. We treat this as an interface trade-off, not as evidence that adaptation is required by the runtime.

**Natural versus Injected Safety.** The natural L4 runs contain no measured stale-write-through events under the current stale-reuse statistic, so they cannot establish a reduction in that rate. Safety claims instead come from the Guard’s conditional construction and must be complemented by controlled corruption tests reporting invalid-action rejection, replay consistency, and unauthorized write-through.

## Analysis and Limitations

We analyze the theoretical and empirical characteristics of MemPatch across three epistemic categories to clearly define its boundaries.

### System Characteristics and Boundaries

**Guaranteed by Construction.** Several key safety properties are guaranteed by the structural design of MemPatch:

- **Complete Mediation and Authorization:** Under the mediated deployment boundary, every persistent-memory write is filtered by  $G_F$ ; actions outside the legal language are not committed.
- **Referential Integrity:** Predicates enforce that target and replacement identifiers must exist in the public view.
- **Replayability and Noninterference:** Given a complete certificate, deterministic replay yields the same post-state, and untargeted records are unchanged.

**Observed Empirically.** On the L4 evaluation split, we observe the following:

- **Accuracy and Mediation Trade-offs:** MemPatch improves state reliability and guarantees a schema-valid projected response, but it does not dominate decision or evidence accuracy. A rejected patch is a no-commit outcome, not proof that the pre-existing state is safe.
- **Backbone Sensitivity:** Benefits are largest when direct generation fails to express the state contract; stronger native structured-output behavior narrows the gap.

**Not Guaranteed.** The system does not guarantee:

- **Semantic Truth:** The Guard verifies structural preconditions and evidence grounding, but cannot verify if a proposed fact is historically or semantically true.
- **Proposal Completeness:** The module is bounded by the model’s generation capacity; if the proposer omits a valid update, the system cannot infer it.
- **Universal Transfer:** Performance depends on the frozen backbone’s capacity to follow action-proposal formatting.

### Limitations

Several limitations define the current scope of our work:



- **Synthetic Ontology and NL Entailment:** The transition ontology in MemPatch-Bench simplifies organic language variability, and incomplete natural-language entailment means the symbolic Guard cannot verify semantic truth.
- **Template-Family Generalization:** Evaluation is conducted over template-family strata rather than arbitrary real-world conversational databases, which might limit external validity.
- **Perturbation Basis:** The seven controlled operators are an auditable diagnostic basis, not an exhaustive model of organic failures; nonlinear interactions may not be identifiable from the local response matrix.
- **Proposal Boundedness:** The runtime cannot recover omitted semantic proposals if the neural model fails to propose them.
- **Incomplete Organic Validation:** The present study uses controlled template families. External validity requires deployment traces, naturally occurring corrections, and downstream write-through measurements.

Future work should investigate organic traces, online multi-agent revision, and learned proposers whose training objective directly balances semantic recall with authorized commitment quality.

## Conclusion

We introduced MemPatch-Bench to expose a failure hidden by answer-only evaluation: a model may use new evidence correctly while leaving an obsolete persistent record licensed for reuse. We then presented MemPatch, a plug-in mediator that compiles model outputs into typed patches and separates semantic proposal from deterministic authorization, commit, and audit. Matched frozen-backbone experiments show a substantial gain in record-state reliability and uniformly valid projected responses, together with model-dependent raw action validity and explicit trade-offs on decision and evidence channels. The central claim is therefore not that a symbolic layer makes a model semantically infallible. It is that persistent-memory mutation should be an explicit, evidence-grounded, replayable operation rather than an unstructured side effect of generation.

## References

Besta, M.; Blach, N.; Kubicek, A.; Gerstenberger, R.; Podstawski, M.; Gianinazzi, L.; Gajda, J.; Lehmann, T.; Niewiadomski, H.; Nyczyk, P.; and Hoefler, T. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(16): 17682–17690.

Chhikara, P.; Khant, D.; Aryan, S.; Singh, T.; and Yadav, D. 2025. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. ArXiv:2504.19413 [cs.CL].

Hu, Y.; Wang, Y.; and McAuley, J. 2026. Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions. ArXiv:2507.05257 [cs.CL].

Kang, J.; Ji, M.; Zhao, Z.; and Bai, T. 2025. Memory OS of AI Agent. ArXiv:2506.06326 [cs.AI].

Li, Y.; Guo, W.; Zhang, L.; Xu, R.; Huang, M.; Liu, H.; Xu, L.; Xu, Y.; and Liu, J. 2026. Locomo-Plus: Beyond-Factual Cognitive Memory Evaluation Framework for LLM Agents. ArXiv:2602.10715 [cs.CL].

Lu, M.; Wu, M.; Liu, F.; Xu, J.; Li, W.; Wang, H.; Hu, Z.; Ding, Y.; Sun, Y.; Lu, J.; and Zhang, Y. 2026. Choosing How to Remember: Adaptive Memory Structures for LLM Agents. ArXiv:2602.14038 [cs.AI].

Luo, Y.; Gao, R.; Teng, L.; Wen, X.; Jiang, J.; Zhang, Q.; Sun, Y.; Zhang, S.; Feng, J.; Liu, T.; Zhang, W.; and Pei, D. 2026. Graph of States: Solving Abductive Tasks with Large Language Models. ArXiv:2603.21250 [cs.AI].

Madaan, A.; Tandon, N.; Gupta, P.; Hallinan, S.; Gao, L.; Wiegrefe, S.; Alon, U.; Dziri, N.; Prabhume, S.; Yang, Y.; Gupta, S.; Majumder, B. P.; Hermann, K.; Welleck, S.; Yazdanbakhsh, A.; and Clark, P. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems (NeurIPS)*.

Maharana, A.; Lee, D.-H.; Tulyakov, S.; Bansal, M.; Barbieri, F.; and Fang, Y. 2024. Evaluating Very Long-Term Conversational Memory of LLM Agents. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 13851–13870.

Packer, C.; Wooders, S.; Lin, K.; Fang, V.; Patil, S. G.; Stoica, I.; and Gonzalez, J. E. 2024. MemGPT: Towards LLMs as Operating Systems. ArXiv:2310.08560 [cs.AI].

Shinn, N.; Cassano, F.; Berman, E.; Gopinath, A.; Narasimhan, K.; and Yao, S. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS)*.

Shutova, A.; Olenina, A.; Vinogradov, I.; and Sinitsin, A. 2026. Evaluating Memory Structure in LLM Agents. ArXiv:2602.11243 [cs.LG].

Tan, H.; Zhang, Z.; Ma, C.; Chen, X.; Dai, Q.; and Dong, Z. 2025. MemBench: Towards More Comprehensive Evaluation on the Memory of LLM-based Agents. *Findings of the Association for Computational Linguistics: ACL 2025*, 19336–19352.

Wang, G.; Xie, Y.; Jiang, Y.; Mandlekar, A.; Xiao, C.; Zhu, Y.; Fan, L.; and Anandkumar, A. 2023. Voyager: An Open-Ended Embodied Agent with Large Language Models. ArXiv:2305.16291 [cs.AI].

Wang, Y.; Zhang, Z.; Chi, M.; Yu, K.; Li, Y.; Peng, M.; Tong, B.; Zhang, C.; Zhou, Y.; and Li, J. 2026. EvoMemBench: Benchmarking Agent Memory from a Self-Evolving Perspective. ArXiv:2605.18421 [cs.CL].

Wu, D.; Ji, Z.; Kawatkar, A.; Kwan, B.; Gu, J.-C.; Peng, N.; and Chang, K.-W. 2026. LongMemEval-V2: Evaluating Long-Term Agent Memory Toward Experienced Colleagues. ArXiv:2605.12493 [cs.CL].

Wu, D.; Wang, H.; Yu, W.; Zhang, Y.; Chang, K.-W.; and Yu, D. 2025. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. *International Conference on Learning Representations (ICLR)*.

Xu, B.; Chen, Y.; Fang, J.; Zhong, R.; Yao, Y.; Zhu, Y.; Du, L.; and Deng, S. 2026. StructMem: Structured Memory

for Long-Horizon Behavior in LLMs. ArXiv:2604.21748 [cs.CL].

Xu, W.; Liang, Z.; Mei, K.; Gao, H.; Tan, J.; and Zhang, Y. 2025. A-MEM: Agentic Memory for LLM Agents. *Advances in Neural Information Processing Systems (NeurIPS)*.

Yao, S.; Yu, D.; Zhao, J.; Shafran, I.; Griffiths, T. L.; Cao, Y.; and Narasimhan, K. 2023a. Tree of Thoughts: Deliberate Problem Solving with Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)*.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2023b. ReAct: Synergizing Reasoning and Acting in Language Models. *International Conference on Learning Representations (ICLR)*.

Yu, Y.; Yao, L.; Xie, Y.; Tan, Q.; Feng, J.; Li, Y.; and Wu, L. 2026. Agentic Memory: Learning Unified Long-Term and Short-Term Memory Management for Large Language Model Agents. ArXiv:2601.01885 [cs.CL].

Zhang, G.; Jiang, W.; Wang, X.; Behr, A.; Zhao, K.; Friedman, J.; Chu, X.; and Anoun, A. 2026. Adaptive Memory Admission Control for LLM Agents. ArXiv:2603.04549 [cs.AI].

Zhou, A.; Yan, K.; Shlapentokh-Rothman, M.; Wang, H.; and Wang, Y.-X. 2024. Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models. *Proceedings of the 41st International Conference on Machine Learning*, 62138–62160.